# Lab 4 — Chat History

Compiled from the Lab-4 chat content provided in this conversation

**Important:** This document contains the chat-history material you provided here. It is not an official ChatGPT account export.

Perfect! ■ **Task 1 is now working.**

So your current status is:

- ■ Opposite-direction reversal fixed - ■ Wall collision works - ■ Self-collision works - ■ Snake stops when collision happens - ■ Proper Game Over screen — **not done yet** - ■ Replay/difficulty — **not done yet** - ■ Sounds — **not done yet**

Now we move to **Task 2**.

## TASK 2 — Game Over Screen

The README requires:

> Add a screen that displays the final score once the snake hits a wall or itself, then gracefully waits for input instead of just printing to the console.

Currently your terminal says:

```
Game over! Final score: X
```

That's what we're going to replace with an actual screen.

## First, don't change anything.

We'll use your current working code as the starting point.

Send me this prompt next in our ChatGPT conversation:

Task 2 prompt

Task 1 is working correctly now.

I want to implement Task 2 from the README.

Task 2: "Add a screen that displays the final score once the snake hits a wall or itself, then gracefully waits for input instead of just printing to the console."

Requirements:

1. Show a clear "GAME OVER" message on the Pygame screen. 2. Show the final score. 3. Stop the snake from moving after collision. 4. Wait for player input instead of immediately closing the game. 5. Keep the existing Task 1 collision fixes working. 6. Do not implement Task 3 or Task 4 yet. 7. Make the minimum necessary changes.

Based on my current main.py and game_engine.py, tell me exactly which files need to change and provide the complete corrected code for each changed file.

**Send that as your next prompt here.**

Then I'll help you apply the returned code **without breaking Task 1**.

Task 1 is working correctly now.

I want to implement Task 2 from the README.

Task 2:

"Add a screen that displays the final score once the snake hits a wall or itself, then gracefully waits for input instead of just printing to the console."

Requirements:

1. Show a clear "GAME OVER" message on the Pygame screen. 2. Show the final score. 3. Stop the snake from moving after collision. 4. Wait for player input instead of immediately closing the game. 5. Keep the existing Task 1 collision fixes working. 6. Do not implement Task 3 or Task 4 yet. 7. Make the minimum necessary changes.

Based on my current main.py and game_engine.py, tell me exactly which files need to change and provide the complete corrected code for each changed file.

Yes. Since Task 1 is working, **Task 2 only needs changes to `main.py` and `game/game_engine.py`**. `snake.py` should stay as it is.

The idea is:

```
Snake hits wall/self
        ↓
game_over = True
        ↓
Game Over screen appears
        ↓
Final score displayed
        ↓
Game waits for a key
        ↓
Player presses a key
        ↓
Game closes
```

For Task 2, we are **not adding replay yet**. Replay will be Task 3.

## 1. Replace `game/game_engine.py`

Open:

```
game/game_engine.py
```

Press **Ctrl + A**, delete everything, and paste:

```
import pygame
from .snake import Snake
from .food import Food

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20

        self.grid_width = width // self.cell_size
        self.grid_height = height // self.cell_size

        self.snake = Snake(
            self.grid_width // 2,
            self.grid_height // 2,
            self.cell_size
        )

        self.food = Food(
            self.grid_width,
            self.grid_height,
            self.cell_size
        )

        self.score = 0

        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over_font = pygame.font.SysFont("Arial", 60)
        self.game_over_score_font = pygame.font.SysFont("Arial", 35)
        self.game_over_instruction_font = pygame.font.SysFont(
            "Arial",
            25
        )

        self.moves_per_second = 8
        self._frame_counter = 0

        self.game_over = False

    def handle_keydown(self, key):
        # When game is over, any key can continue/exit.
        if self.game_over:
            return

        # Arrow keys and WASD
        if key in (pygame.K_UP, pygame.K_w):
            self.snake.set_direction(0, -1)

        elif key in (pygame.K_DOWN, pygame.K_s):
            self.snake.set_direction(0, 1)

        elif key in (pygame.K_LEFT, pygame.K_a):
            self.snake.set_direction(-1, 0)

        elif key in (pygame.K_RIGHT, pygame.K_d):
            self.snake.set_direction(1, 0)

    def handle_input(self):
        pass

    def update(self):
        # Stop updating the snake after Game Over.
        if self.game_over:
            return

        self._frame_counter += 1
```

```
        frames_per_move = max(
            1,
            60 // self.moves_per_second
        )

        if self._frame_counter < frames_per_move:
            return

        self._frame_counter = 0

        # Move the snake
        self.snake.move()

        # Check wall collision
        if self.snake.collides_with_wall(
            self.grid_width,
            self.grid_height
        ):
            self.game_over = True
            return

        # Check self collision
        if self.snake.collides_with_self():
            self.game_over = True
            return

        # Check food collision
        if self.snake.head_rect().colliderect(
            self.food.rect()
        ):
            self.snake.grow()
            self.score += 1
            self.food.respawn(self.snake.body)

    def render(self, screen):
        # Draw food
        pygame.draw.rect(
            screen,
            RED,
            self.food.rect()
        )

        # Draw snake
        for rect in self.snake.segment_rects():
            pygame.draw.rect(
                screen,
                GREEN,
                rect
            )

        # Draw score
        score_text = self.font.render(
            f"Score: {self.score}",
            True,
            WHITE
        )

        screen.blit(score_text, (10, 10))

        # Draw Game Over screen
        if self.game_over:
            game_over_text = self.game_over_font.render(
                "GAME OVER",
                True,
                WHITE
            )

            final_score_text = self.game_over_score_font.render(
                f"Final Score: {self.score}",
                True,
                WHITE
            )

            instruction_text = self.game_over_instruction_font.render(
```

```
                "Press any key to exit",
                True,
                WHITE
            )

            screen.blit(
                game_over_text,
                game_over_text.get_rect(
                    center=(self.width // 2, self.height // 2 - 70)
                )
            )

            screen.blit(
                final_score_text,
                final_score_text.get_rect(
                    center=(self.width // 2, self.height // 2)
                )
            )

            screen.blit(
                instruction_text,
                instruction_text.get_rect(
                    center=(self.width // 2, self.height // 2 + 60)
                )
            )
```

Then press:

**Ctrl + S**

## 2. Replace `main.py`

Open:

```
main.py
```

Press **Ctrl + A** and replace everything with:

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake - Pygame Version")

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)


def main():
    running = True

    while running:
```

```
            SCREEN.fill(BLACK)

            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False

                if event.type == pygame.KEYDOWN:
                    if engine.game_over:
                        # Task 2: exit after the player acknowledges
                        # the Game Over screen.
                        running = False
                    else:
                        engine.handle_keydown(event.key)

            engine.handle_input()
            engine.update()
            engine.render(SCREEN)

            pygame.display.flip()
            clock.tick(FPS)

        pygame.quit()


    if __name__ == "__main__":
        main()
```

Press:

**Ctrl + S**

## 3. Test Task 2

Run:

```
    python main.py
```

Play normally.

Make the snake hit:

## Wall

or

## Its own body

You should now see:

```
            GAME OVER

          Final Score: 4

        Press any key to exit
```

The snake should **stop moving**.

Then press a key.

The program should close.

## Why we changed `main.py`

Previously, when Game Over happened, the game just continued running its loop while `engine.update()` did nothing.

Now:

```
if engine.game_over:
    running = False
```

happens **when the player presses a key after Game Over**.

So the game stays on the Game Over screen until the player presses something.

## Why we changed `game_engine.py`

This:

```
if self.game_over:
    return
```

prevents the snake from continuing to move.

And this:

```
if self.game_over:
```

inside `render()` displays:

- **GAME OVER** - **Final Score** - **Press any key to exit**

## After testing

Your status should be:

- ■ Task 1 — collision/direction fix - ■ Task 2 — Game Over screen - ■ Task 3 — replay + Easy/Medium/Hard - ■ Task 4 — sounds

**Test Task 2 now.** If the Game Over screen appears and pressing a key closes the game, Task 2 is complete. Then we'll move to **Task 3**, where instead of simply exiting, we'll add the difficulty/replay menu.

task two is working

Perfect! ■ **Task 2 is complete.**

Your current progress:

| Task | Status | | ---------- | ------- | | Task 1 — Collision fixes | ■ Done | | Task 2 — Game Over screen + final score | ■ Done | | Task 3 — Replay + difficulty selection | ■ Next | | Task 4 — Sound feedback | ■ Later |

Before moving on, **save your work** with `Ctrl + S`.

Also, if you haven't already, keep your **BEFORE video** and later we'll make the **AFTER video** for submission.

## Next: Task 3

Task 3 will change the current:

> `Press any key to exit`

into a proper menu such as:

```
   GAME OVER

  Final Score: 5

PLAY AGAIN?

1 - EASY
2 - MEDIUM
3 - HARD
Q - EXIT
```

We'll do **only Task 3** next and leave Task 4 (sound) untouched.

## Chat memory full

Start a new chat with an automatic summary of this conversation, or upgrade to continue here with expanded memory in every chat.

Continue in a new chat

svgUpgrade to Plus